

机器学习 *GraphRAG* 知识问答 *Agent*

剩余四天冲刺执行方案

先完成科研闭环，再增加一个技术增强点

消融实验 · 人工评分 · 误差分析 · 技术增强 · 报告与答辩

当前项目状态	<i>Day 1~Day 5</i> 已完成：核心系统、真实 <i>LangGraph</i> 、三路检索、 <i>Verifier</i> 、 <i>Streamlit</i> 和冻结评测集均已具备
现有知识库	6 个 <i>scikit-learn</i> 官方页面；180 个 <i>Chunk</i> ；50 个实体；100 条 <i>approved</i> 关系并绑定真实证据
当前评测	冻结保留集 40 题；39 题决策正确；4 道无答案题全部正确拒答
剩余时间	4 天
执行优先级	科研实验与报告保底 > 单个技术增强 > 演示包装
推荐增强	优先确认真实 <i>LLM</i> 生成器；若已具备，则做 <i>Sparse + Dense + Graph</i> 三源融合与图谱感知重排序
版本	<i>V1.0 · 2026-07-22</i>

总原则：今天先冻结一个绝对可交付的 *v1.0* 基线版本，完成消融、人工评分、误差分析和报告骨架；随后仅在独立分支增加一个技术增强。增强失败时，项目仍可按 *v1.0* 完整交付。

目录

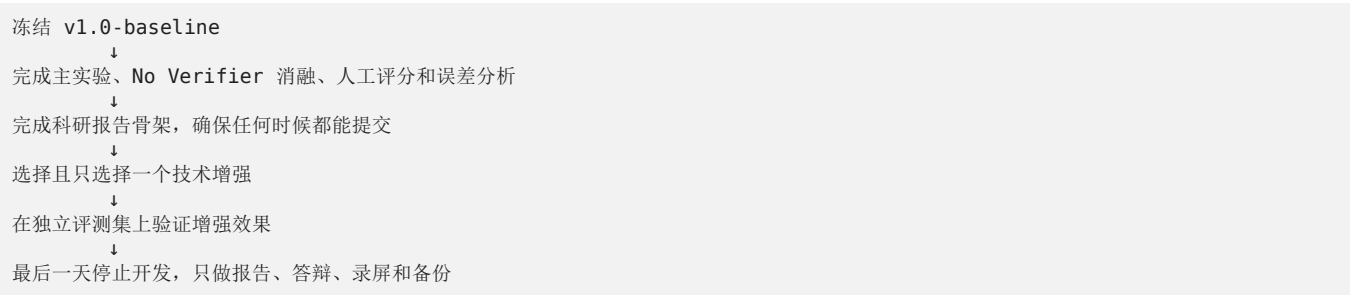
- 0. 结论与四天总策略
 - 1. 当前进度审计与项目成熟度判断
 - 2. 今天必须完成的版本冻结
 - 3. Day 1: 消融实验、人工评分与误差分析
 - 4. 技术增强的选择决策树
 - 5. 推荐增强 A: 可插拔真实 LLM 结构化生成
 - 6. 推荐增强 B: Sparse + Dense + Graph 三源融合
 - 7. 增强实验与防止测试集泄漏
 - 8. Day 2~Day 4 逐时实施计划
 - 9. 实验指标、表格与可视化方案
 - 10. 科研报告详细写作方案
 - 11. AdaBoost 错误案例的标准分析
 - 12. 延迟、后端和技术表述修正
 - 13. 答辩演示与常见追问
 - 14. 风险止损与最终验收清单
- 附录 A. Git 冻结与分支命令
- 附录 B. 消融实验配置模板
- 附录 C. 新增评测集格式
- 附录 D. 技术增强代码骨架

0. 结论与四天总策略

0.1 现在应该先做什么

明确结论：不要现在直接在主分支继续堆功能。先完成 Day 6 的核心科研工作并冻结 v1.0，再在独立分支增加一个技术增强点。

0.2 四天的正确顺序



0.3 为什么不能先继续加功能

- 1. 当前核心系统已完成，项目主要风险已经从“做不出来”转为“科研论证不完整”。
- 2. 冻结保留集已经运行一次，继续根据错误改动再重复使用会造成测试泄漏。
- 3. 没有消融、人工正确性评分和误差分析，即使功能更多，也难以证明模块真正有效。
- 4. 技术增强可能引发依赖、模型下载、结果波动或现有功能回归，必须有稳定版本作为保底。
- 5. 报告和答辩需要真实实验表、失败案例和严谨表述，这些比再加一个界面功能更能提升得分。

1. 当前进度审计与项目成熟度判断

1.1 已完成的核心能力

模块	当前状态	评价
项目骨架与统一 Schema	已完成：路径、Pydantic 数据结构、GraphRepository 接口齐全	工程基础扎实
文本知识库	6 页官方文档、164 Sections、180 Chunks	范围符合冻结方案
向量检索	离线 Chroma + TF-IDF，可中文检索英文证据	稳定可复现，但语义能力可增强
知识图谱	50 实体、100 approved 关系、全部绑定真实 Chunk	闭环完整
图后端	NetworkX/Neo4j 可切换	容错设计有亮点
检索 workflow	Graph / Vector / Hybrid + 动态路由	课题核心已完成
LangGraph	真实 StateGraph 已启用	符合课题要求
Verifier	引用、路径、主题属性、一次重试与拒答	具备科研创新点
前端	Streamlit 四区、桌面与移动端已验收	演示条件成熟
评测	开发/演示/先导/冻结集分离；冻结集仅运行一次	实验设计严谨

1.2 当前结果应该如何表述

现有结果	正确表述	禁止表述
39/40 决策正确	冻结集决策准确率 97.5%	问答准确率 97.5%
4/4 无答案题拒答	无答案题拒答准确率 100%，但样本仅 4 道	系统不会产生幻觉
引用率 97.5%	39 道输出包含引用	证据忠实度 97.5%
平均耗时 1.48 ms	本地 workflow/规则路径的热启动测量，需拆分解释	完整在线 LLM 问答仅需 1.48 ms

成熟度判断：项目已经达到“可提交且有优秀潜力”的阶段。接下来的提升主要来自实验严谨性、技术增强的对照结果、报告表达和稳定演示，而不是扩大知识库。

2. 今天必须完成的版本冻结

2.1 冻结 v1.0-baseline

推荐 *Git* 命令

```
git status
git add .
git commit -m "release: freeze complete GraphRAG QA baseline"
git tag v1.0-baseline
git switch -c feature/technical-enhancement
```

2.2 必须归档的文件

- ☐ `reports/evaluation_final.json`。
- ☐ `reports/evaluation_pilot_postfix.json`。
- ☐ `reports/streamlit_desktop_final.png`。
- ☐ `reports/streamlit_mobile_final.png`。
- ☐ `reports/streamlit_refuse.png`。
- ☐ 冻结时的 `requirements.txt` 和 `pip freeze` 输出。
- ☐ 冻结时的 `config/settings.yaml`、`sources.yaml` 和 `ontology.yaml`。
- ☐ 冻结时的 `entities.csv`、`relations.csv`、`chunks_snapshot.jsonl`。

2.3 创建发布清单

```
reports/releases/v1.0-baseline/
├── evaluation_final.json
├── evaluation_pilot_postfix.json
├── screenshots/
├── config_snapshot/
├── data_statistics.json
├── environment.txt
├── git_commit.txt
└── release_notes.md
```

2.4 `release_notes.md` 应记录

- 系统功能范围和明确不包含的内容。
- 当前评测结果与唯一已知失败 `T-DF-01`。
- 当前向量检索使用 `TF-IDF` 表示。
- `LangGraph`、`Neo4j/NetworkX`、`Streamlit` 的运行状态。
- 已知限制：语义检索能力、测试集规模、规则生成或 `LLM` 生成模式。

3. Day 1：消融实验、人工评分与误差分析

3.1 主实验与消融矩阵

方法	文本检索	图检索	动态路由	Verifier	目的
Direct LLM	否	否	否	否	无知识库基线
Vector RAG	是	否	固定 <i>Vector</i>	可关闭	普通 <i>RAG</i> 基线
Graph Only	否/仅绑定证据	是	固定 <i>Graph</i>	可关闭	结构关系基线
Proposed	是	是	是	是	完整方案
No Verifier	是	是	是	否	验证器消融，必做
No Router（可选）	是	是	固定 <i>Hybrid</i>	是	路由消融，有时间再做

3.2 先检查 Direct LLM 是否真实可运行

若项目当前主要使用离线规则生成器，应将 *Direct LLM* 标为“需要联网模型接口的实验配置”；如果无法稳定调用真实 LLM，不要伪造 *Direct LLM* 结果。可以在报告中保留 *Vector*、*Graph*、*Proposed* 和 *No Verifier* 四组可复现实验，并把 *Direct LLM* 作为补充实验。

3.3 人工回答正确性评分

分数	定义	示例
0	错误、答非所问、关键结论相反或错误拒答	可回答问题被拒答；错误说明算法用途
1	部分正确，但遗漏关键点或存在轻微不准确	对比题只说一个维度
2	关键答案点完整、无实质错误、表达清晰	覆盖 <i>gold_answer_points</i>

3.4 忠实度评分

分数	定义
0	关键结论无证据，或引用与结论明显不匹配
1	部分 <i>Claim</i> 有证据，仍包含 <i>unsupported</i> 专业事实
2	所有关键 <i>Claim</i> 均由当前文本证据或图路径支持

3.5 推荐人工评分文件

reports/human_scoring.csv

```
question_id,method,correctness_score,faithfulness_score,
hallucination,over_refusal,notes,reviewer
```

```
T-DF-01,Proposed,0,2,0,1,
"AdaBoost 定义题证据不足导致保守拒答",student
```

3.6 必须计算的指标

指标	计算方式	说明
Decision Accuracy	决策正确题数 / 总题数	路由后的 <i>pass/refuse</i> 是否正确
Answer Correctness	人工得分总和 / (2×题数)	真正的回答质量
Evidence Faithfulness	忠实度得分总和 / (2×题数)	不能用引用率代替
Retrieval Recall@5	<i>gold Chunk</i> 进入 <i>Top-5</i> 的题数 / 可答题	检索模块指标
Path Validity	真实路径数 / 输出路径数	图检索可信度
Refusal Accuracy	正确拒答 / 无答案题	注明样本数
Hallucination Rate	含无证据事实的回答 / 总回答	越低越好
Over-refusal Rate	可回答但拒答 / 可答题	反映保守性
Latency	分阶段和端到端时间	区分本地和 LLM 模式

3.7 误差分析最低要求

- ☐ 至少分析 8~10 个案例，不只分析唯一错误。
- ☐ 包含真正错误、险些失败、过度拒答和低质量证据案例。
- ☐ 每个案例标注错误发生阶段：实体链接、路由、图检索、文本检索、融合、生成、验证。
- ☐ 每个案例写出原因、证据和可行改进，不能只写“模型能力不足”。
- ☐ 保留 *T-DF-01* 原结果，不针对它修改后重新声称冻结集成绩。

4. 技术增强的选择决策树

4.1 先检查当前答案生成器

```
当前 answer_generator 是否真实调用 LLM?  
├-- 否：优先做增强 A  
│   可插拔 LLMAnswerGenerator + OfflineRule fallback  
└-- 是：优先做增强 B  
    Sparse + Dense + Graph 三源融合与图谱感知重排序
```

4.2 两个增强的优先级

情况	首选增强	原因
目前答案主要由规则模板拼接	增强 A: 真实 LLM 结构化生成	课题是知识问答 <i>Agent</i> ，真实生成能力优先于检索锦上添花
已经稳定调用真实 LLM，并有结构化输出	增强 B: <i>Sparse + Dense + Graph</i>	当前 <i>TF-IDF</i> 是主要技术短板，三源融合有清晰实验价值

情况	首选增强	原因
网络或模型下载不稳定	保留 <i>v1.0</i> ，不强行增强	稳定、可复现和按时交付优先

4.3 选择规则

1. 只选一个增强，不并行开发两个。
2. 增强代码必须放独立分支或独立 *backend*，不替换 *v1.0*。
3. 开发时间上限 4~6 小时；超过即停止。
4. 增强必须有对应对照实验，否则不算科研贡献。
5. 增强不允许使用原冻结集调参。

5. 推荐增强 A：可插拔真实 LLM 结构化生成

5.1 适用条件

如果当前 *answer_generator* 主要依赖离线规则和图关系模板，则应优先增加真实 LLM 生成器，同时保留规则生成器作为离线 *fallback*。这一增强能让项目更符合“知识问答 *Agent*”的定位。

5.2 统一接口

```
from typing import Protocol

class AnswerGenerator(Protocol):
    def generate(
        self,
        query: str,
        retrieval_result: RetrievalResult,
    ) -> AnswerPayload:
        ...
```

5.3 两个实现

```
src/agent/generators/
├── base.py
├── llm_generator.py
├── offline_rule_generator.py
└── factory.py
```

5.4 配置

```
answer_generator:
  backend: llm
  fallback: offline_rule
  temperature: 0.0
  max_retries: 1
  timeout_seconds: 30
```

5.5 LLM 输出模型

```
class Claim(BaseModel):
    claim: str
    evidence_ids: list[str]

class AnswerPayload(BaseModel):
    answer: str
    claims: list[Claim]
    graph_paths: list[str]
    unsupported_claims: list[str]
```

5.6 技术要点

- 使用 *Pydantic/JSON Schema* 约束输出。
- *Prompt* 中明确禁止使用模型记忆补充专业事实。
- 输出解析失败最多重试一次。
- 调用失败、超时或密钥缺失时自动回退规则生成器。
- 前端显示 *generator_backend*，便于答辩说明。
- *Verifier* 对 *LLM* 输出执行与规则输出相同的验证。

5.7 对应实验

方法	检索	生成	验证
Rule Generator	Proposed Retrieval	离线规则	Verifier
LLM Generator	完全相同	真实 LLM	Verifier
LLM No Verifier	完全相同	真实 LLM	关闭

5.8 成功标准

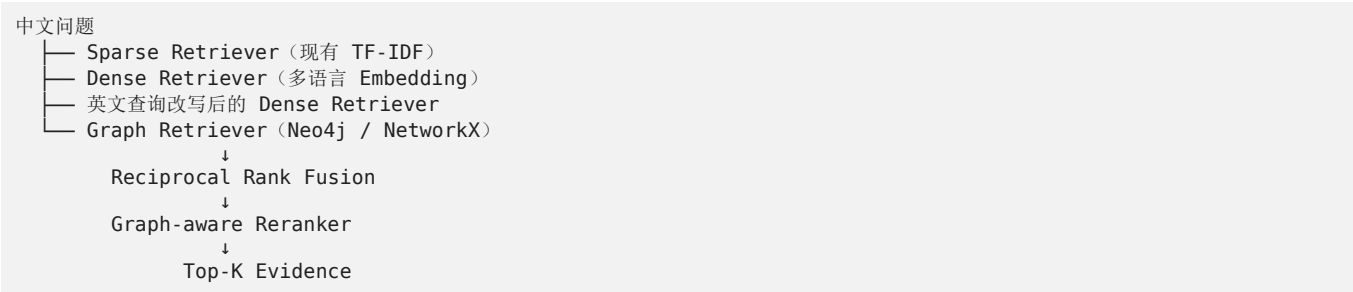
- ☐ 结构化输出成功率≥95%。
- ☐ 答案可读性和完整性优于规则生成。
- ☐ 忠实度不低于规则模式，或通过 *Verifier* 显著修正。
- ☐ 网络故障时可以无感回退到离线模式。

6. 推荐增强 B : Sparse + Dense + Graph 三源融合

6.1 适用条件

如果当前已经稳定调用真实 *LLM*，则最值得增强的是检索层。当前 *Chroma* 使用 *TF-IDF* 特征，优点是离线可复现，但对同义改写和语义表达的泛化有限。可以增加多语言 *Dense Embedding*，并与现有稀疏检索和图谱路径融合。

6.2 增强架构



6.3 RRF

```
RRF(d) = Σ 1 / (k + rank_r(d))
```

推荐:

- k = 60
- sparse Top-10
- dense zh Top-10
- dense en Top-10
- 合并后保留 12 条, 再重排到 6~8 条

6.4 图谱感知重排序

```
final_score =  
    0.45 * normalized_rrf  
    + 0.20 * entity_overlap  
    + 0.15 * graph_path_relevance  
    + 0.10 * direct_evidence_bonus  
    + 0.10 * heading_match
```

特征	含义	实现方式
entity_overlap	问题实体与 <i>Chunk</i> 实体重合	别名归一后计算覆盖率
graph_path_relevance	<i>Chunk</i> 是否绑定本次图路径	<i>chunk_id</i> 是否存在于 <i>path evidence</i>
direct_evidence_bonus	是否直接回答目标属性	定义/优势/局限关键词或结构标签
heading_match	标题路径是否包含实体或问题属性	标准化字符串匹配

6.5 Dense 模型选择原则

- 优先使用已能在本机加载的多语言模型，不在最后四天尝试超大模型。
- 模型下载和构建超过 2 小时立即止损。
- 保留现有 *Sparse Retriever*，不替换；*Dense* 失败时自动退回 *v1.0*。
- 缓存 *Embedding*，避免每次运行重新计算。

6.6 对应消融

方法	<i>Sparse</i>	<i>Dense</i>	<i>Graph</i>	融合/重排
<i>Sparse Only</i>	是	否	否	无
<i>Dense Only</i>	否	是	否	无

方法	Sparse	Dense	Graph	融合/重排
Sparse + Dense	是	是	否	RRF
Full Enhanced	是	是	是	RRF + Graph-aware

6.7 成功标准

- ☐ 在新 *extension holdout* 上 *Retrieval Recall@5* 有实际提升。
- ☐ 复杂改写和中英混合术语题明显改善。
- ☐ 端到端正确性或忠实度提高，而不是只增加复杂度。
- ☐ 延迟增加可接受，并在报告中如实说明。

7. 增强实验与防止测试集泄漏

7.1 原冻结集的地位

不可变规则：原 *final_questions.jsonl* 的 40 题结果必须保留为 *v1.0* 唯一一次冻结评测。任何针对 *T-DF-01* 或阈值的后续修改，都不能重新把该数据集称为未见最终测试。

7.2 新建 *extension holdout*

题型	建议数量	重点
复杂改写定义题	5	同义表达、长句、术语省略
中英混合术语题	4	<i>Random Forest</i> 、 <i>F1</i> 、 <i>SVM</i> 等
多跳关系题	5	实体—方法族—任务—指标
对比与原因题	5	多来源证据融合
无答案/错误前提	4	拒答和事实校验
合计	23	编码前冻结

7.3 数据文件

```
data/evaluation/extension_holdout.jsonl
data/evaluation/extension_readme.md
reports/extension_baseline.json
reports/extension_enhanced.json
reports/extension_comparison.csv
```

7.4 调试边界

- 增强开发只允许使用 *dev* 和 *pilot*。
- *extension_holdout* 在代码完成后运行一次。
- 运行 *extension_holdout* 后不再调整增强参数。

- 若确实需要继续调整，需再创建一套新的测试集并明确标注。

8. Day 2~Day 4 逐时实施计划

8.1 第 1 天：科研保底版本

时间	任务	输出
09:00-09:30	冻结 v1.0、Git tag、归档配置和结果	v1.0-baseline
09:30-11:30	实现/检查实验配置，运行主实验	main_results
13:00-14:30	运行 No Verifier 消融	ablation_no_verifier
14:30-17:00	40 题人工正确性与忠实度评分	human_scoring.csv
17:00-18:00	计算指标与分题型统计	metrics_summary.csv
19:00-20:30	整理 8~10 个错误/边界案例	error_analysis.md
20:30-21:30	完成报告方法与实验章节骨架	report_outline.docx/md

第 1 天结束标准：即使后面技术增强完全失败，项目也已经具有完整实验、误差分析和报告骨架，可以按时提交。

8.2 第 2 天：只做一个技术增强

时间	任务	输出
09:00-09:30	确认答案生成器状态并选择增强 A 或 B	decision.md
09:30-10:00	冻结 extension_holdout	23 题新保留集
10:00-12:00	完成增强模块最小实现	backend prototype
13:00-15:00	接入统一接口和配置开关	可切换实现
15:00-16:00	运行现有回归测试	pytest 全通过
16:00-17:00	在 dev/pilot 上调试	初步结果
17:00	止损检查：未稳定则回退 v1.0	go/no-go
19:00-21:00	若通过，补单元测试、日志和 README	增强候选版

增强止损：累计开发 4~6 小时仍无法稳定运行，立即停止。禁止牺牲第 3、4 天的报告和答辩时间。

8.3 第 3 天：增强评测与科研报告

时间	任务	输出
09:00-10:00	运行 extension baseline	baseline JSON

时间	任务	输出
10:00–11:00	运行 <i>enhanced</i> ，测试集只运行一次	<i>enhanced JSON</i>
11:00–12:00	计算增强前后指标和代价	<i>comparison.csv</i>
13:00–14:30	生成实验表和图	<i>figures/tables</i>
14:30–16:00	写技术增强方法与算法描述	报告第 5 章
16:00–17:30	写主实验、消融、增强实验与误差分析	报告第 7 章
19:00–20:30	整理系统截图、图谱截图和典型案例	报告素材
20:30–21:30	报告第一次完整通读	<i>draft v1</i>

8.4 第 4 天：停止开发，只做交付

时间	任务	输出
09:00–10:30	从干净环境按 <i>README</i> 复现	复现记录
10:30–12:00	报告定稿、参考文献、图表编号	<i>final report</i>
13:00–14:30	制作 8~10 页答辩 <i>PPT</i>	<i>slides</i>
14:30–16:00	固定演示脚本和问题顺序	<i>demo script</i>
16:00–17:30	完整彩排 3 次并计时	<i>rehearsal notes</i>
19:00–20:00	录制 3~5 分钟演示视频	<i>demo video</i>
20:00–21:00	最终打包、双重备份、 <i>Git release</i>	<i>submission package</i>

9. 实验指标、表格与可视化方案

9.1 主结果表

方法	<i>Correctness</i> ↑	<i>Faithfulness</i> ↑	<i>Recall@5</i> ↑	<i>Path Validity</i> ↑	<i>Refusal Acc.</i> ↑	<i>Hallucination</i> ↓	<i>Latency</i> ↓
<i>Direct LLM</i>	待填	待填	—	—	待填	待填	待填
<i>Vector RAG</i>	待填	待填	待填	—	待填	待填	待填
<i>Graph Only</i>	待填	待填	待填	待填	待填	待填	待填
<i>Proposed</i>	待填	待填	待填	待填	待填	待填	待填

9.2 消融表

方法	动态路由	<i>Verifier</i>	<i>Correctness</i> ↑	<i>Faithfulness</i> ↑	<i>Over-refusal</i> ↓	<i>Hallucination</i> ↓
<i>Proposed</i>	是	是	待填	待填	待填	待填
<i>No Verifier</i>	是	否	待填	待填	待填	待填

方法	动态路由	Verifier	Correctness↑	Faithfulness↑	Over-refusal↓	Hallucination↓
No Router（可选）	否	是	待填	待填	待填	待填

9.3 推荐图表

- 各方法总体正确性、忠实度柱状图。
- 按题型的 *Proposed vs Vector RAG* 正确性图。
- *No Verifier* 对幻觉率、拒答准确率的影响图。
- 检索 *Recall@5* 与端到端正确性的对应图。
- 错误类型分布图：实体、检索、融合、生成、验证。
- 增强前后 *extension holdout* 指标对比图。

9.4 结果结论模板

主实验应回答“完整方案在哪些题型优于基线”；消融应回答“去掉 *Verifier* 后具体损失什么”；增强实验应回答“新增模块是否带来统计上可观察的收益，以及代价是什么”。不要只写 *Proposed* 最好。

10. 科研报告详细写作方案

10.1 推荐章节

章节	本项目内容	建议页数
第 1 章 绪论	背景、问题、研究目标、RQ1~RQ3、贡献	2~3
第 2 章 相关技术	RAG、知识图谱、GraphRAG 思想、LangGraph、证据验证	3~4
第 3 章 数据与知识库	六页来源、Chunk、Chroma、图谱本体与证据绑定	4~5
第 4 章 总体架构	统一接口、三路检索、状态图和后端切换	3~4
第 5 章 核心方法	路由、Hybrid、生成、Verifier；新增增强点	5~7
第 6 章 系统实现	模块、Streamlit、异常回退、工程测试	3~4
第 7 章 实验	数据划分、基线、指标、主实验、消融、增强、误差分析	6~8
第 8 章 总结	结论、局限和展望	1~2

10.2 贡献点推荐表述

1. 构建了一个以 *scikit-learn* 官方文档为文本证据、以预定义知识图谱为结构知识的机器学习领域问答知识库。
2. 设计了基于 *LangGraph* 的查询理解、动态三路检索、证据融合、答案生成和验证回退流程。

- 3. 设计了引用合法性、*Claim* 覆盖和图路径真实性联合验证机制，并支持无证据问题拒答。
- 4. 实现了 *Neo4j* 与 *NetworkX* 统一图仓储接口，提高了系统离线可运行性和容错性。
- 5. 通过严格分割的开发、先导和冻结测试集，完成基线、消融和错误分析。

10.3 增强点写法

如果实施增强 *A*，将贡献写成“可插拔结构化 *LLM* 生成与离线规则回退”；如果实施增强 *B*，将贡献写成“稀疏—稠密—图谱三源融合与图谱感知重排序”。不要同时把两个未完成模块写成贡献。

10.4 实验章节必须说明

- ☐ 首套 40 题作为 *pilot* 用于发现实现缺口，不能作为无泄漏最终结果。
- ☐ 重新构建的 40 题 *final* 保留集与开发/先导集题面不重合。
- ☐ *final* 冻结后仅运行一次，未根据结果继续调参。
- ☐ 唯一错误 *T-DF-01* 被保留并用于过度拒答分析。
- ☐ 若新增 *extension_holdout*，也要说明在增强完成后仅运行一次。

11. AdaBoost 错误案例的标准分析

11.1 案例描述

项目	内容
问题 ID	<i>T-DF-01</i>
问题类型	定义解释题
期望行为	回答 <i>AdaBoost</i> 的基本定义并引用 <i>Ensemble</i> 页面
实际行为	<i>Verifier</i> 判定证据不足，系统保守拒答
错误类型	<i>Over-refusal</i> / 检索充分度判断过严

11.2 可能原因

- 1. 实体链接正确，但直接定义 *Chunk* 在稀疏检索中的排序不够高。
- 2. 检索结果包含 *AdaBoost* 相关信息，但未命中 *Verifier* 要求的“直接定义属性”。
- 3. 标题路径或实体匹配奖励不足，泛化段落排在定义段落之前。
- 4. *Verifier* 阈值偏保守，导致可回答问题被拒答。

11.3 改进方向

- 定义题增加实体名称与标题路径匹配奖励。
- 对定义意图优先检索实体对应页面和 *Section*。
- 将“定义充分度”与其他解释题的阈值分开。
- 使用 *Dense Retrieval* 改善同义描述召回。

11.4 报告中的价值

科研价值：这个案例证明 *Verifier* 在降低幻觉的同时可能产生过度拒答，体现了安全性与可回答率之间的权衡。保留该错误比把结果修成 *40/40* 更有研究说服力。

12. 延迟、后端和技术表述修正

12.1 延迟必须拆分

```
latency = {
  "normalize_ms": ...,
  "entity_link_ms": ...,
  "graph_retrieval_ms": ...,
  "vector_retrieval_ms": ...,
  "fusion_ms": ...,
  "generation_ms": ...,
  "verification_ms": ...,
  "end_to_end_ms": ...
}
```

测量条件	报告写法
离线规则生成 + 热启动	本地离线 workflow 延迟
真实 LLM API	在线完整问答延迟，单独统计
首次加载 <i>Chroma</i> /图谱	冷启动延迟
后续连续查询	热启动延迟

12.2 Neo4j 与 NetworkX 的正确定位

主演示优先使用 *Neo4j*。*NetworkX* 不应被描述为主数据库，而应描述为统一 *GraphRepository* 接口下的离线降级后端，用于提升系统容错性和无数据库环境下的可复现性。

12.3 GraphRAG 表述

准确表述：本项目是基于预定义知识图谱的轻量化混合 *GraphRAG / Knowledge-Graph-Enhanced RAG*，借鉴图结构与文本证据结合的思想，但未实现 *Microsoft GraphRAG* 的完整社区检测和全局摘要流水线。

13. 答辩演示与常见追问

13.1 三分钟演示顺序

时间	演示	讲解重点
0:00-0:25	系统首页与数据统计	小型但完整：180 <i>Chunk</i> 、50 实体、100 关系
0:25-0:55	定义题	<i>Vector/Dense</i> 与官方引用

时间	演示	讲解重点
0:55-1:25	关系题	Graph 路径和证据回指
1:25-1:55	原因/对比题	Hybrid 与 LangGraph 路由
1:55-2:20	错误前提或知识库外题	Verifier 与拒答
2:20-2:45	实验表	主实验、No Verifier 和错误分析
2:45-3:00	技术增强	仅展示有实验结果的增强

13.2 常见追问

问题	回答要点
为什么先做实验再加功能？	核心系统已完成，科研项目必须证明模块价值；先冻结确保可交付。
为什么不修复唯一错误再跑一次？	冻结集已经被观察，继续调参会造成测试泄漏；错误被保留作科研分析。
97.5% 是什么准确率？	是 pass/refuse 决策准确率，不等同于回答正确率；回答质量另有人工评分。
技术增强是什么？	只回答实际完成并评测的 A 或 B，不夸大。
NetworkX 是不是主图数据库？	不是；Neo4j 为主，NetworkX 是统一接口下的离线降级后端。
为什么有拒答？	证据不足时拒答可减少无依据结论，但会带来过度拒答，需要实验权衡。

14. 风险止损与最终验收清单

14.1 增强风险

风险	止损信号	处理
LLM 接口不稳定	2 小时仍无法稳定结构化输出	保留规则生成器，不做增强 A
Dense 模型无法下载	2 小时仍失败	保留 Sparse；不继续换多个模型
增强导致回归	现有 14+ 测试失败或拒答失控	回退 v1.0 分支
实验耗时过长	第 3 天上午仍未完成	缩小 extension 集，不动原 final
报告时间不足	第 3 天晚上报告未成型	停止增强，专注交付

14.2 最终验收

- ☐ v1.0-baseline tag 可独立运行。
- ☐ 主实验和 No Verifier 消融结果已保存。
- ☐ 40 题人工正确性和忠实度评分完成。
- ☐ 误差分析至少 8 个案例。

- ☐ 唯一冻结集错误未被掩盖或重复调参。
- ☐ 技术增强最多一个，且有独立实验结果。
- ☐ 报告清楚区分决策准确率、回答正确性、引用率和忠实度。
- ☐ 延迟说明区分离线/在线、冷启动/热启动。
- ☐ *Neo4j* 主演示稳定，*NetworkX* 降级可用。
- ☐ *README* 可从干净环境复现。
- ☐ 答辩 *PPT*、演示脚本、视频和双重备份完成。

最后提醒：接下来四天最能提高项目质量的不是功能数量，而是：严谨实验、真实误差分析、一个有对照结果的技术增强，以及稳定清楚的报告和答辩。

附录 A. Git 冻结与分支命令

```
git status
git add .
git commit -m "release: freeze complete GraphRAG QA baseline"
git tag v1.0-baseline

git switch -c feature/technical-enhancement

# 查看冻结版本
git show v1.0-baseline --stat

# 增强失败后回到保底版本
git switch main
git reset --hard v1.0-baseline
```

附录 B. 消融实验配置模板

```
experiments:
  direct_llm:
    retrieval_mode: none
    verifier_enabled: false

  vector_rag:
    retrieval_mode: vector
    verifier_enabled: false

  graph_only:
    retrieval_mode: graph
    verifier_enabled: false

  proposed:
    retrieval_mode: adaptive
    verifier_enabled: true

  no_verifier:
    retrieval_mode: adaptive
    verifier_enabled: false

  no_router:
    retrieval_mode: hybrid
    verifier_enabled: true
```

附录 C. 新增评测集格式

```
{
  "id": "EXT-MH-01",
  "question": "哪些模型属于树模型，并可用于分类？",
  "question_type": "multi_hop",
  "answerable": true,
  "gold_entities": ["Tree Models", "Classification"],
  "gold_paths": [
    "Decision Tree-BELONGS_TO->Tree Models",
    "Decision Tree-SOLVES->Classification"
  ],
  "gold_chunk_ids": ["..."],
  "gold_answer_points": ["决策树", "随机森林等相关树模型"]
}
```

附录 D. 技术增强代码骨架

D.1 生成器工厂

```
def create_answer_generator(settings):
    backend = settings.answer_generator.backend

    if backend == "llm":
        try:
            return LLMAnswerGenerator(settings)
        except Exception:
            return OfflineRuleAnswerGenerator(settings)

    return OfflineRuleAnswerGenerator(settings)
```

D.2 RRF

```
def reciprocal_rank_fusion(result_lists, k=60):
    scores = {}
    by_id = {}

    for results in result_lists:
        for rank, item in enumerate(results, start=1):
            chunk_id = item.chunk_id
            scores[chunk_id] = scores.get(chunk_id, 0.0) + 1.0 / (k + rank)
            by_id[chunk_id] = item

    return sorted(
        by_id.values(),
        key=lambda item: scores[item.chunk_id],
        reverse=True,
    )
```

D.3 阶段耗时

```
from time import perf_counter

start = perf_counter()
# run stage
latency_ms = (perf_counter() - start) * 1000
```